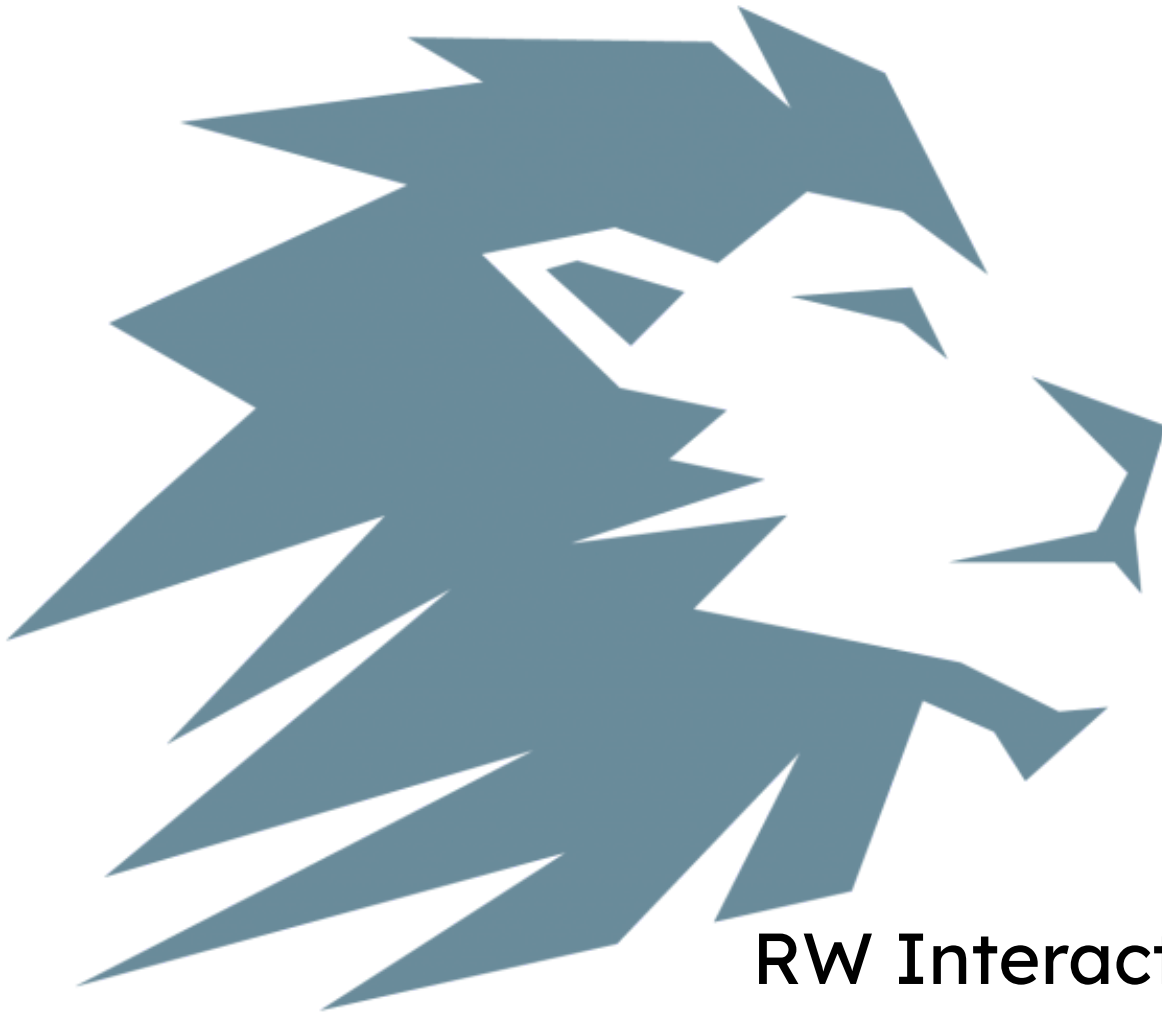




RW Interactive

Piquet - Ambiente Local



02 - Ambiente local

Laravel Sail (Docker). O `docker-compose.yml` na raiz do **backend** levanta:

Serviço	Imagem	Porta (default)
laravel.test	sail-8.3/app (build local)	\${APP_PORT:-80}, \${VITE_PORT:-5173}, 8080 fixo (Reverb)
mysql	mysql/mysql-server:8.0	\${FORWARD_DB_PORT:-3306}
redis	redis:alpine	\${FORWARD_REDIS_PORT:-6379}
meilisearch	getmeili/meilisearch:latest	\${FORWARD_MEILISEARCH_PORT:-7700}
mailpit	axllent/mailpit:latest	1025 (SMTP) + \${FORWARD_MAILPIT_DASHBOARD_PORT:-8025} (UI)
meilisearchUi	riccoxie/meilisearch-ui:latest	8090

Requisitos: Docker + Docker Compose. **Não precisas de PHP nem de Composer instalados;** o arranque é feito por um container descartável.



Passo 0 - Layout do Diretório

O `composer.json` do backend traz o SDK de pagamentos como repositório **path**:

```
"repositories": {  
  "payshop-sdk": {  
    "type": "path",  
    "url": "../payshop-sdk",  
    "options": { "symlink": true }  
  }  
}
```

../payshop-sdk - o SDK é IRMÃO do backend, não filho.

O `docker-compose.yml` monta o mesmo caminho ('../payshop-sdk:/var/www/payshop-sdk').

```
piquet/      ← a diretoria em que trabalhas  
├── backend/  
└── payshop-sdk/ ← IRMÃO. Se estiver noutro sítio, o composer install falha.
```

```
mkdir piquet && cd piquet  
git clone https://gitlab.rwinteractive.net/piquet/backend.git  
git clone https://gitlab.rwinteractive.net/piquet/payshop-sdk.git  
git -C backend checkout production # minúsculas  
git -C payshop-sdk checkout production # minúsculas
```

Nota: `main` está desatualizado em **TODOS** os repositórios. Trabalha sempre a partir da branch de produção.

A caixa do nome da branch importa, e não é cosmética. No `app-vendor` existem duas branches que diferem só na maiúscula, e não são a mesma coisa:



ref	o que é REALMENTE
app-vendor/production (minúscula)	a app de vendor - é esta
app-vendor/Production (maiúscula)	a app de cliente, mal colocada. Divergiu em 2024-09-30.



Passo 1 - Composer Install

Nota: Problema do ovo e da galinha: o `docker-compose.yml` faz build a partir de `./vendor/laravel/sail/runtimes/8.3` que **só existe depois do `composer install`** e o `composer install` precisa de PHP. Solução: um container descartável.

```
cd piquet
```

```
docker run --rm \
  --user "$(id -u):$(id -g)" \
  -e COMPOSER_HOME=/tmp/composer \
  -v "$(pwd)":/opt -w /opt/backend \
  laravelsail/php83-composer:latest \
  composer install --ignore-platform-reqs --no-interaction --no-scripts
```

NOTAS:

1. **Porquê `-v "$(pwd)":/opt` e não a diretoria do backend:** o composer precisa de ver o `../payshop-sdk`, que está **fora** do backend. Montar só o backend faz o install falhar.
2. **Porquê PHP 8.3:** o `backend/composer.json` declara `php: ^8.2`, mas o `payshop-sdk` exige `^8.3`. Em PHP 8.2 o install não resolve. O `^8.2` é uma **declaração errada** (DEP-012); a produção corre em 8.3.
3. **Porquê `--user + COMPOSER_HOME` (mudou nesta revisão):** sem eles o container corre como **root** e deixa o `vendor/` inteiro a pertencer ao root, obrigando a um `sudo chown -R` a seguir. **Com `--user`, verificado a 2026-07-17: o `vendor/` fica teu (`exit`**



0, sem root, sem **sudo**). O **COMPOSER_HOME=/tmp/composer** é preciso porque um utilizador não-root não tem home com permissão de escrita nesta imagem. O **sudo chown -R** que estava aqui antes deixou de ser necessário.



Passo 2 - O Ficheiro de Ambiente

```
cd backend && cp .env.example .env
```

NOTAS

APP_PORT: já vem certo (8000)

O `.env.example` traz **APP_PORT=8000**. O `docker-compose.yml` publica o Reverb num **8080** fixo ("**8080:8080**"). Se puseres **APP_PORT=8080**, os dois querem o mesmo porto do host e o Docker **não dá erro**: o container fica **Up** mas **sem portos publicados**, site inalcançável em silêncio. Regra: **nunca 8080**. Confirma sempre com `docker port backend-laravel.test-1` (tem de devolver 3 linhas).

Alterações obrigatórias

O `.env.example` aponta tudo para **127.0.0.1**. **Dentro do container, 127.0.0.1 é o próprio container** e tem de passar a nomes de serviço do compose:

No <code>.env.example</code>	Tem de ficar	Porquê
<code>DB_HOST=127.0.0.1</code>	<code>DB_HOST=mysql</code>	nome do serviço no compose
<code>REDIS_HOST=127.0.0.1</code>	<code>REDIS_HOST=redis</code>	idem
<code>MAIL_HOST=127.0.0.1</code> <code>MAIL_PORT=587</code>	<code>MAIL_HOST=mailpit</code> <code>MAIL_PORT=1025</code>	● obrigatório - o login do backoffice manda um código 2FA por email; sem Mailpit não entras (ver Passo 6)
<code>MEILISEARCH_HOST=http://127.0.0.1:7700</code>	<code>MEILISEARCH_HOST=http://meilisearch:7700</code>	idem

(**APP_PORT** já vem **8000** no `.env.example` - não mexer.)

Confirmar que existem (o `.env.example` não os traz):

`WWWGROUP=1000`



```
WWWUSER=1000
DB_DATABASE=piquet
DB_USERNAME=sail
DB_PASSWORD=password
VITE_PORT=15173                # ver abaixo
FORWARD_DB_PORT=13306          # evita colidir com um MySQL local
FORWARD_REDIS_PORT=16379
FORWARD_MEILISEARCH_PORT=17700
FORWARD_MAILPIT_DASHBOARD_PORT=18025
```

NOTAS

1. failed to bind host port 0.0.0.0:5173/tcp: address already in use
Qualquer outro projeto Vite a correr rouba a porta 5173. **Define sempre o VITE_PORT.**
 2. O backend arranca, migra, semeia e serve o backoffice com `PAYSHOP_SDK_*`,
`INVOICE_XPRESS_*`, `TWILIO_*` e `GOOGLE_MAPS_GEOCODING_API_KEY` **todos vazios.**
 3. **MOCK_SMS:** o `.env.example` traz **MOCK_SMS=false**. Em local mete **MOCK_SMS=true** para não dependeres de SMS reais. **Nunca true em produção.** Parte o login por telemóvel em silêncio.
 4. **Segredos:** não se pedem por chat nem por email.
-



Passo 3 - Docker Up

```
./vendor/bin/sail up -d
```

Nota: Na primeira vez que faz build da imagem e puxa MySQL/Redis/Meilisearch/Mailpit, demora um pouco

```
docker ps --filter name=backend- --format "{{.Names}}\t{{.Status}}"
```

esperado: laravel.test, mysql, redis, meilisearch, mailpit, meilisearchUi - todos Up

Nota: Cuidado com o estado **Up**. Como se viu no Passo 2, um container pode estar **Up** e não ter portos nenhuns. **Confirma sempre com `docker port backend-laravel.test-1`**.

Tem de devolver três linhas (**80→8000**, **8080→8080**, **15173→15173**). Se devolver vazio, é o **APP_PORT**.

O MySQL demora ~40s a ficar **healthy**. Espera antes do passo seguinte:

```
docker inspect backend-mysql-1 --format '{{.State.Health.Status}}' # → healthy
```



Passo 4 - Chaves, assets e migrações

Pela ordem. Nenhum destes passos é opcional - sem eles, `/backoffice/login` dá 500.

```
./vendor/bin/sail artisan key:generate      # APP_KEY (vem vazia do .env.example)
./vendor/bin/sail artisan jwt:secret        # ⚠️ JWT_SECRET vem VAZIA e nada avisa
./vendor/bin/sail artisan migrate           # verificado: corre limpo, 0 erros
./vendor/bin/sail npm install
./vendor/bin/sail npm run build             # ⚠️ sem isto, public/build não existe → 500
```

NOTAS

1. O `JWT_SECRET` vazio e o `public/build` em falta são a causa do 500 no backoffice e não dão erro nenhum até abrires a página.
 2. O `jwt:secret` imprime a chave no terminal. É descartável e só local, mas não coles a saída deste comando em lado nenhum.
-



Passo 5 - Seed da Base de Dados

db:seed

Semeia 11 tipos de serviço, 10 áreas de operação, 38 géneros.

```
./vendor/bin/sail artisan db:seed --force
```

Criar o utilizador de acesso

```
./vendor/bin/sail artisan db:seed --class=UserSeeder --force
```

O UserSeeder está COMENTADO no DatabaseSeeder

(`#$this->callOnce(UserSeeder::class);`), por isso o `db:seed` normal **não cria nenhum utilizador** - e sem utilizador com role não entras no backoffice. **Tem de ser chamado à mão.**

Email	Password	Role
<code>admin@example.com</code>	<code>password</code>	<code>admin</code> → entra no <code>/backoffice</code>
<code>user@vendor.com</code>	<code>password</code>	vendor
<code>user@customer.com</code>	<code>password</code>	cliente

(Passwords definidas no `UserSeeder.php`. **Confirmadas** nesta verificação via `Hash::check`. Só para desenvolvimento local.)

Só **admin/super-admin** entram no backoffice. O `user@vendor.com` e o `user@customer.com` são **rejeitados no login** (verificado - `Login.php:112` barra antes de sequer enviar o código).



Passo 6 - Verificação

Nota: Um **200** na página de login não prova que consegues entrar. A página rende antes de haver utilizador, antes de haver role, e antes do `canAccessPanel()` decidir.

1. a app responde

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8000/up # 200
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8000/backoffice/login # 200
```

2. o que interessa: as credenciais servem E o utilizador entra mesmo no painel

```
./vendor/bin/sail artisan tinker --execute="
    \$u = App\Models\User::where('email', 'admin@example.com')->first();
    echo 'password ok: ' . (Illuminate\Support\Facades\Hash::check('password',
        \$u->password) ? 'YES': 'NO').PHP_EOL;
    echo 'roles: ' . \$u->getRoleNames()->implode(',').PHP_EOL;
    echo 'canAccessPanel:
        ' . (\$u->canAccessPanel(Filament\Facades\Filament::getPanel('backoffice'))) ?
        'YES': 'NO').PHP_EOL;
"
```

Resultado:

password ok: YES

roles: admin

canAccessPanel: YES

Passo 6.1 - o login do backoffice tem 2FA por email (o código vai para o Mailpit)

O fluxo no browser é:

1. Ir a <http://localhost:8000/backoffice/login>, mete `admin@example.com / password`.
2. O sistema valida as credenciais e **envia um código de 6 dígitos por email**. Em local esse email cai no **Mailpit**, não numa caixa real.
3. Abre o **Mailpit** (<http://localhost:18025>), abre o email *"Your backoffice access code"*, copia o código de 6 dígitos.



4. Mete o código no desafio → entras no backoffice.

```
# prova de que o email do código chega ao Mailpit:
./vendor/bin/sail artisan tinker --execute="
\$u = App\Models\User::where('email','admin@example.com')->first();
app(App\Support\BackofficeTwoFactor::class)->start(\$u, false);    // simula o
    envio do login
echo 'código enviado - vê o Mailpit'.PHP_EOL;
"

curl -s http://localhost:18025/api/v1/messages | python3 -c "import sys,json;
    m=json.load(sys.stdin)['messages'][0]; print('para:', m['To'][0]['Address'],
    '| assunto:', m['Subject'])"
# → para: admin@example.com | assunto: Your backoffice access code
```

Nota: Regras do código ([config/backoffice-2fa.php](#)): 6 dígitos, expira em **10 min, 5 tentativas**, reenvio com **60 s** de cooldown. Sem tabela/migração - o hash vive na sessão. **Se o email não aparecer no Mailpit:** o `MAIL_*` não está a apontar para o `mailpit` (Passo 2).

Onde	URL
Backoffice	http://localhost:8000/backoffice/login
Mailpit (emails + código 2FA)	http://localhost:18025
Meiliseach UI	http://localhost:8090



Resumo

```
mkdir piquet && cd piquet
git clone https://gitlab.rwinteractive.net/piquet/backend.git
git clone https://gitlab.rwinteractive.net/piquet/payshop-sdk.git
git -C backend checkout production && git -C payshop-sdk checkout production

docker run --rm --user "$(id -u):$(id -g)" -e COMPOSER_HOME=/tmp/composer \
  -v "$(pwd)":/opt -w /opt/backend laravelsail/php83-composer:latest \
  composer install --ignore-platform-reqs --no-interaction --no-scripts

cd backend && cp .env.example .env
# → editar o .env conforme a tabela do Passo 2 (OBRIGATÓRIO):
#   hosts 127.0.0.1 → nomes de serviço (mysql/redis/mailpit/meilisearch), MAIL para
#   mailpit,
#   VITE_PORT=15173, WWWUSER/WWWGROUP, DB_DATABASE/USERNAME/PASSWORD. (APP_PORT já
#   vem 8000.)

./vendor/bin/sail up -d
docker port backend-laravel.test-1 # tem de devolver 3 linhas. Vazio = APP_PORT
errado.

./vendor/bin/sail artisan key:generate
./vendor/bin/sail artisan jwt:secret
./vendor/bin/sail artisan migrate
./vendor/bin/sail npm install && ./vendor/bin/sail npm run build

./vendor/bin/sail artisan db:seed --force # corre limpo (seeder já
corrigido)
./vendor/bin/sail artisan db:seed --class=UserSeeder --force

# → http://localhost:8000/backoffice/login (admin@example.com / password)
# → depois vai ao Mailpit http://localhost:18025 buscar o código 2FA de 6 dígitos
```

Parar / limpar:

```
./vendor/bin/sail down # parar
./vendor/bin/sail down -v # parar e APAGAR os volumes (base de dados incluída)
```